

Universidad
Rey Juan Carlos

Sistemas Operativos

[PRÁCTICA I – PROGRAMACIÓN C]

JOSÉVÍCTOR GARCÍA LLORENTE



TABLA DE CONTENIDO

Autor	2
Descripción del Código	3
Diseño del Código	3
Principales Funciones	5
Casos de Prueba	7
Comentarios Personales	9
PROBLEMAS ENCONTRADOS	9
CRÍTICAS CONSTRUCTIVAS	9
PROPUESTA DE MEJORAS	10
EVALUACIÓN DEL TIEMPO DEDICADO	10



Autor

Nombre: José Víctor.

Apellidos: García Llorente.

Curso: Tercero.

Titulación: Doble grado en ingeniería del software + matemáticas.



Descripción del Código

Diseño del Código

- ESTRUCTURA DEL PROGRAMA TEST

El programa test en primer lugar hace una llamada a la función *main*, la cual se ejecutaría nada más comenzar. Se pasan como argumentos *argc* que es un entero que representa el número de parámetros introducidos y *argv* que contiene los argumentos pasados al llamar al programa desde la shell en nuestro caso.

Lo siguiente que se hace es la comprobación de argumentos mediante un *switch*. En caso de que *argc* sea 2 significa que se ha pasado un solo argumento por la shell. Pero si es 3, se habrán pasado 2. Cualquier otro caso se contempla en la opción *default*.

Ya hemos comprobado la validez de *argc*, por lo que lo siguiente será verificar en el caso de que tengamos uno o dos argumentos, si los mismos son válidos. Para ello, dentro de cada *case* tenemos la comprobación de si cada elemento de *argv[1]* es válido y comprobamos si lo que se ha introducido por teclado se corresponde a cada función en cuestión, precedido por un guión tal y como se pide. En caso de que todo sea correcto, se llama a la función correspondiente con *argv[2]*, pero transformándolo primero en un entero usando *atoi*, ya que por defecto los elementos de *argv* son cadenas de caracteres.

- ALGORITMOS UTILIZADOS PARA CADA FUNCIONALIDAD PEDIDA

Función *head*: Se pueden diferenciar tres partes esencialmente. La primera es de declaración de variables, así como de reserva de memoria para las *N* líneas que se quieren imprimir más 1. Esta última se usará de forma auxiliar para que una vez hayamos leído las *N* líneas que nos interesan, se pase a un segundo bucle de tipo *while* que leerá líneas que realmente se perderán, pero las *N* líneas que queríamos guardar se quedan guardadas hasta que se pulse Ctrl+D. En este momento se pasa a un tercer bucle que imprime por pantalla la secuencia de frases guardadas.

Función *tail*: Al igual que la anterior, se hace un bucle que reserva espacio para *N+1* líneas y lee las *N* primeras. A continuación entra en uno nuevo de tipo *while* que antes de cada iteración, guarda en la posición *N+1* del array la nueva línea que se ha introducido. Dentro del *while* hay un *for* que itera sobre cada posición del array, enlazando cada posición más antigua con la siguiente más nueva (Diagrama 1). Por último, se imprimen las líneas por pantalla.

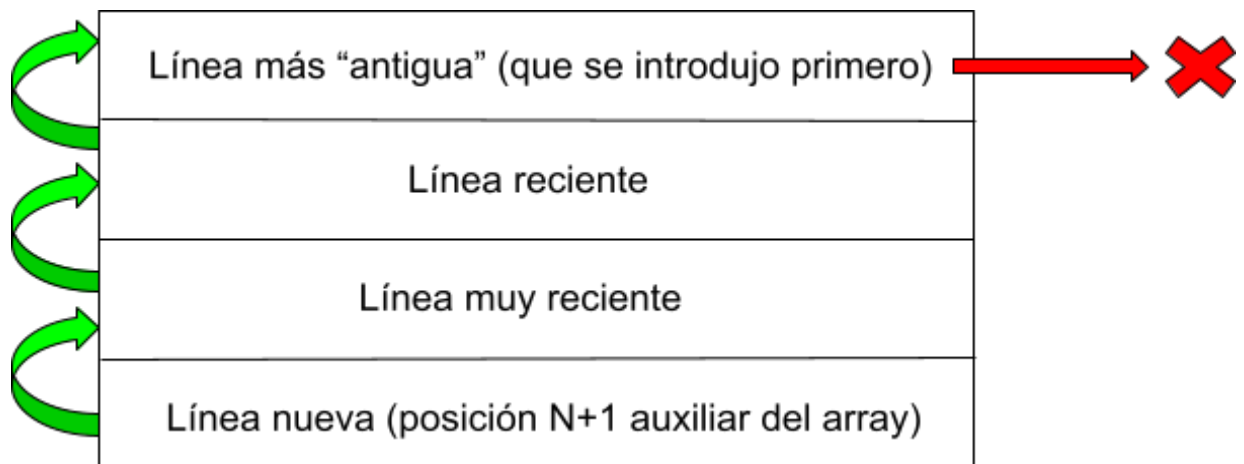


Diagrama 1: Funcionamiento del bucle for anidado en el while (función tail). Para un array de 3+1 posiciones.

Función *longlines*: Tiene prácticamente la misma estructura que las funciones descritas anteriormente (reserva de memoria, bucle *while* e impresión por pantalla) con la excepción de que dentro del bucle *while* se observa una función la cual he llamado *ordenar*. Esta función cuyo propio nombre indica lo que hace, es realmente la que ordena el array de strings al completo en función de su longitud. *Ordenar* recibe dos parámetros: el array con las cadenas de caracteres y un entero *max* que indica el número de posiciones totales, es decir ($max = N$) y mediante un bucle *for*, recorre cada posición del array de cadenas. Para hacerlo más eficiente, se comprueba si la longitud de la cadena en la posición *j* es menor a la nueva línea introducida que se encuentra en la posición *N+1* (ya que $max = N$). Si es así, mediante el algoritmo de ordenación de la burbuja, se sustituyen los valores de la posición *j* por los de *max* y los de *max* por los de *j*, y así repetidas veces hasta la finalización del bucle *for* (y por supuesto del bucle *while*, mediante el cual el usuario proporciona las líneas por entrada estándar). (Diagrama 2).

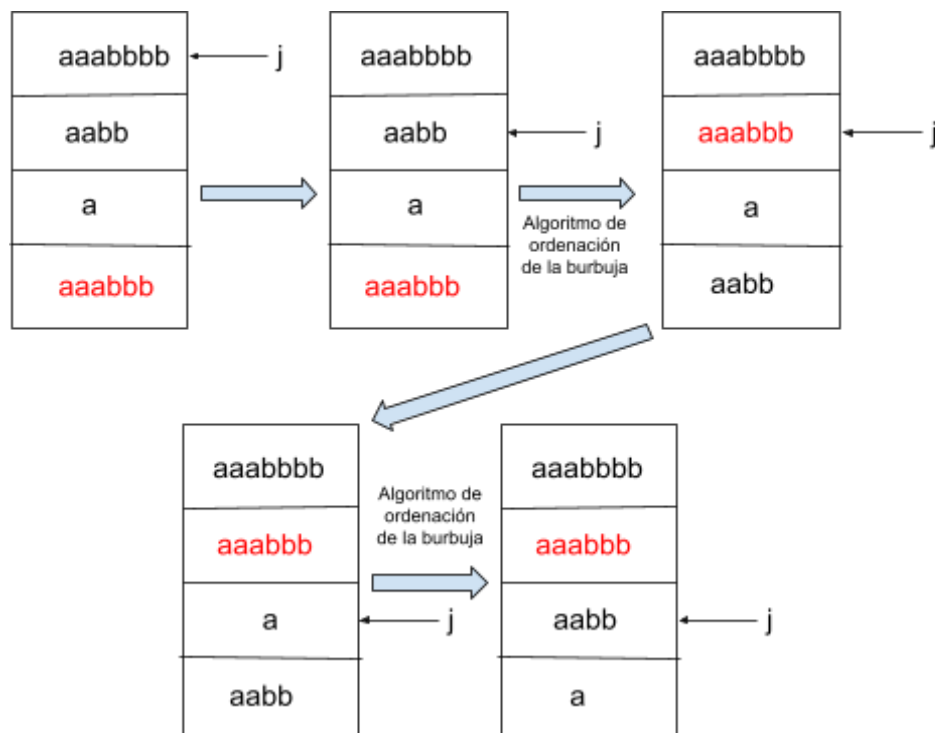




Diagrama 2: Funcionamiento del algoritmo de ordenación (función longlines).

Principales Funciones

	main	Nombre	Tipo	Descripción
Argumentos	Argumento 1	argc	int	Variable de tipo entero que representa el número de argumentos que se le ha proporcionado al programa. Incluye como argumento el nombre del programa
	Argumento 2	argv	char**	Array de cadenas de caracteres. En el caso particular de este programa argv[1] correspondería al nombre de la función que se quiere invocar y argv[2] el número de líneas con el que se quiere trabajar en dicha función. Cabe mencionar argv[0], nombre del programa.
Variables Locales	No existen variables locales a destacar.			
Valor Devuelto			int	0 si la ejecución ha sido exitosa.
Descripción de la Función	Función principal del programa, contiene la estructura principal del programa, las llamadas a las distintas funciones implementadas en la librería y la gestión de argumentos.			

	head	Nombre	Tipo	Descripción
Argumentos	Argumento 1	N	int	Variable de tipo entero. Representa el número de líneas que se mostrarán por pantalla al finalizar su ejecución.
Variables Locales	Variable 1	i	int	Sirve como índice del array de cadenas de caracteres.
	Variable 2	cadena	char**	Array de cadenas de caracteres. Usado para leer las N primeras líneas que se introducen por teclado y operar con una línea adicional.



Valor Devuelto			int	0 si la ejecución ha sido exitosa.
Descripción de la Función	Función de librería. Guarda e imprime las N primeras líneas que se introducen por teclado.			

	tail	Nombre	Tipo	Descripción
Argumentos	Argumento 1	N	int	Variable de tipo entero. Representa el número de líneas que se mostrarán por pantalla al finalizar su ejecución.
Variables Locales	Variable 1	i	int	Sirve como índice del array de cadenas de caracteres.
	Variable 2	cadena	char**	Array de cadenas de caracteres. Usado para leer las N últimas líneas que se introducen por teclado y operar con una línea adicional.
Valor Devuelto			int	0 si la ejecución ha sido exitosa.
Descripción de la Función	Función de librería. Guarda e imprime las N últimas líneas que se introducen por teclado.			

	longlines	Nombre	Tipo	Descripción
Argumentos	Argumento 1	N	int	Variable de tipo entero. Representa el número de líneas que se mostrarán por pantalla al finalizar su ejecución.
Variables Locales	Variable 1	i	int	Sirve como índice del array de cadenas de caracteres.
	Variable 2	cadena	char**	Array de cadenas de caracteres. Usado para leer N líneas que se introducen por teclado, operar con una línea adicional y ordenarlas.
Valor Devuelto			int	0 si la ejecución ha sido exitosa.



Descripción de la Función	Función de librería. Ordena dinámicamente las N líneas que se van introduciendo por entrada estándar. Imprime en orden primero las líneas con mayor longitud y por último las de menor longitud.
---------------------------	--

	ordenar	Nombre	Tipo	Descripción
Argumentos	Argumento 1	max	int	Es igual que la variable N de longlines.
	Argumento 2	cadena	char**	Array de cadenas de caracteres. Igual que la variable cadena de longlines.
Variables Locales	Variable 1	j	int	Sirve como índice del array de cadenas de caracteres.
	Variable 2	aux	char*	Array de caracteres. Sirve como variable auxiliar en el algoritmo de ordenación de la burbuja.
Valor Devuelto			int	0 si la ejecución ha sido exitosa.
Descripción de la Función	Función auxiliar. Usada por longlines. Ordena la cadena de caracteres mediante el algoritmo de ordenación de la burbuja.			

Casos de Prueba

A continuación se muestra una tabla con los casos de prueba más interesantes de cara a la interacción entre la persona y el programa. Los casos en los que la persona hace un correcto funcionamiento del programa han sido obviados, pues han sido probados muchas veces y funcionan bien. Por ejemplo si se llama a cualquier función con un número N de líneas y el usuario introduce x tal que $N \leq x$.

Función	Detalles de la prueba	Salida esperada	Salida	Conclusión y resultado de la prueba
main	Llamada al programa sin argumentos (tan solo poniendo ./test).	El número de argumentos (0) debe ser 1 o 2	El número de argumentos (0) debe ser 1 o 2	Sí funciona
main	Llamada al programa con un solo argumento no válido (ej. ./test abcdefg)	No se reconoce abcdefg como argumento	No se reconoce abcdefg como argumento	Sí funciona



main	Llamada al programa con dos argumentos no válidos (ej. ./test abc xyz)	No se reconoce abc como argumento	No se reconoce abc como argumento	Sí funciona
main	Llamada al programa con más de dos argumentos (ej. ./test abc xyz 123)	El número de argumentos (3) debe ser 1 o 2	El número de argumentos (3) debe ser 1 o 2	Sí funciona
head	Llamada a head con N líneas e introduciendo por teclado un número x de líneas tal que $0 \leq x \leq N$	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas	Sí funciona
tail	Llamada a tail con N líneas e introduciendo por teclado un número x de líneas tal que $0 \leq x \leq N$	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas	Sí funciona
longlines	Llamada a longlines con N líneas e introduciendo por teclado un número x de líneas tal que $0 \leq x \leq N$	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas ordenadas de mayor a menor longitud	Si introducimos 0 líneas, no muestra nada por pantalla. Si introducimos x líneas con $x > 0$, imprime las x líneas ordenadas de mayor a menor longitud	Sí funciona



Comentarios Personales

PROBLEMAS ENCONTRADOS

A lo largo del transcurso de la elaboración de esta práctica ha habido ciertos problemas que me han llevado más tiempo solucionar. A continuación los más relevantes, y que más han bloqueado la elaboración de la librería:

- Control de los mandatos de la shell: Para generar los archivos .o y crear la librería (.a), ciertamente tuve que hacer pruebas previas a parte con una librería muy simple para ver cómo funcionaban los mandatos para crear el fichero `compila.sh` y generar un ejecutable. No llevó demasiado tiempo al final pero fue uno de los problemas a destacar, que no sabía cómo resolver, pero que al final lo conseguí.
- El manejo de punteros en C: Aprender a manejar los punteros ha sido un problema, puesto que cuando se comete un fallo en ejecución las notificaciones de error no son nada descriptivas y encontrar dónde se produjo el fallo es una tarea difícil si no controlas bien tu código.
- Creación de las funciones de la librería: Junto al manejo de punteros, como la librería supone manejar memoria dinámicamente, hay funciones que son más difíciles de implementar. En concreto, he tenido ciertos problemas para implementar longlines ya que ordenar mediante punteros un array no ha sido fácil. A veces ocurrían errores que no sabía dónde se producían, pero al final lo he solucionado analizando bien el código.
- No saber por dónde empezar: Un problema inicial que tuve es no saber por dónde empezar. Dada la gran cantidad de información vista en tan poco tiempo, me surgió el problema de cómo plantear la práctica. Las clases prácticas fueron muy útiles para comprender mejor el problema. Además hacer pequeños programas de prueba me ayudó mucho así como programar por mi cuenta.

CRÍTICAS CONSTRUCTIVAS

En mi opinión el enfoque de esta práctica es muy bueno ya que ayuda a comprender mejor el mundo de los punteros, así como compilar un programa y automatizar la creación de una librería mediante un fichero de shell. Por ello doy la enhorabuena a los profesores.

Sin embargo, creo que sería buena idea quizás no de cara a la práctica, sino a la asignatura en general, subir más ejemplos de ejercicios resueltos, tanto de scripts como de ejercicios en C o de exámenes de otros años ya que siento que el material proporcionado ha sido útil, pero no suficiente.



PROPUESTA DE MEJORAS

Por mi parte las propuestas de mejora serían, tal y como he dicho en el punto anterior, proporcionar más ejemplos para programar scripts de shell y de ejercicios en C ya que los he intentado todos por mí mismo y tras hacerles todos y ver las soluciones de los profesores siento que no han sido suficientes para practicar, o quizás subir exámenes o pruebas de otros años también serían muy útiles.

Simplemente añadir que estoy muy contento con esta asignatura y me parecen muy interesantes los temas que toca, por eso de momento no tengo más propuestas de mejora.

EVALUACIÓN DEL TIEMPO DEDICADO

En cuanto al tiempo dedicado a esta práctica, he de reconocer que ha sido menor del que me esperaba que iba a necesitar en un principio. No ha supuesto una dedicación de tiempo excesivo, pero sí ha sido necesario dedicar bastante tiempo a la comprensión de conceptos que quizás, vistos en teoría parecían sencillos de manejar, pero que ha resultado todo lo contrario.

En particular, la parte que más tiempo ha consumido ha sido la implementación del código fuente de la librería y la comprensión del funcionamiento de los punteros en el lenguaje C, sobre todo en la implementación de la función longlines ya que había que aplicar un algoritmo de ordenación con punteros.